

Tokamak Network - TON Staking 2.5

Audit

Preliminary Comments

CertiK Assessed on Mar 4th, 2025





Certik Assessed on Mar 4th, 2025

Tokamak Network - TON Staking 2.5 Audit

These preliminary comments were prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES

Governance, Staking

ECOSYSTEM

Ethereum (ETH)

METHODS

Formal Verification, Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Delivered on 03/04/2025

KEY COMPONENTS

N/A

CODEBASE

[source](#)[script1](#)[script2](#)[View All in Codebase Page](#)

COMMITTS

[01e198130b178757dd194bd8726a1ab678fca167](#)[8f5048305350b1e606030984421c101dd0d80773](#)[8f5048305350b1e606030984421c101dd0d80773](#)[View All in Codebase Page](#)

Vulnerability Summary



15

Total Findings

11

Resolved

0

Mitigated

0

Partially Resolved

2

Acknowledged

0

Declined

2

Pending

0 Critical

2 Major

3 Medium

6 Minor

4 Informational

0 Discussion

2 Pending

3 Resolved

4 Resolved, 2 Acknowledged

4 Resolved

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.

Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

The impact of the issue is yet to be determined, hence requires further clarifications from the project team.

TABLE OF CONTENTS

TOKAMAK NETWORK - TON STAKING 2.5 AUDIT

Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Findings

[CON-01 : Centralization Risks](#)

[CON-02 : Potential Storage Collision](#)

[SMV-05 : Unused Minted Tokens in `layer2Manager` Contract](#)

[SMV-06 : Incorrect Reward Allocation After Unpause](#)

[SMV-07 : Inconsistent Reward Calculation for Layer2](#)

[CON-04 : Third-Party Dependencies](#)

[DAC-02 : Unused Modifier](#)

[DAC-03 : Violation of Check-Effect-Interaction Pattern](#)

[DAC-04 : Ineffective `onlyMemberContract` modifier](#)

[DAO-01 : Lack of Storage Gap Or NameSpaced Storage Layout in Upgradeable Contract](#)

[DMV-01 : `minDepositGasLimit` Is Not Used](#)

[COT-01 : Functions With `` as Name Prefix Are Not `private` or `internal`](#)

[DAC-05 : Redundant Code Components](#)

[DAC-06 : Unused Internal Function](#)

[OMV-01 : Typos](#)

Appendix

Disclaimer

CODEBASE | TOKAMAK NETWORK - TON STAKING 2.5 AUDIT

Repository

[source](#)

[script1](#)

[script2](#)

Commit

[01e198130b178757dd194bd8726a1ab678fca1678f5048305350b1e606030984421c101dd0d80773](#)

[8f5048305350b1e606030984421c101dd0d80773](#)

AUDIT SCOPE | TOKAMAK NETWORK - TON STAKING 2.5 AUDIT

12 files audited ● 9 files with Pending findings ● 3 files without findings

ID	Repo	File	SHA256 Checksum
● DAO	tokamak-network/ton-staking-v2	contracts/proxy/DAOCommitteeProxy2.sol	62ad03b7b56bdd484a93ff3736d2f27d716874aaa1f9191be902b473deed5893
● DAC	tokamak-network/ton-staking-v2	contracts/dao/DAOCommittee_V1.sol	5a7cdfd85ee838500a5915968d2a8652d9e3f7a3a7ce033ba7e91edaa36f4295
● DOC	tokamak-network/ton-staking-v2	contracts/dao/DAOCommitteeOwner.sol	ade43e95e1de4347152aa0cedf3dc44e3a7f9f9e57853789d381765576e4f950
● CAO	tokamak-network/ton-staking-v2	contracts/dao/CandidateAddOnV1_1.sol	7e7a5608d684c813ed14f7265fbc158a7eff3011bc9cbfde1784b57466f83a35
● SMV	tokamak-network/ton-staking-v2	contracts/stake/managers/SeigManagerV1_3.sol	f7a262a613cbec9e18476bbd12ad4ff3a0468abc6b53f491dd1e252ce8cedf
● DMV	tokamak-network/ton-staking-v2	contracts/stake/managers/DepositManagerV1_1.sol	46a3337d9c0eb8ef811039c72b640b33a389df14672647698751c90893037bee
● LBR	tokamak-network/ton-staking-v2	contracts/layer2/L1BridgeRegistryV1_1.sol	6aa44497580cfb39c52d3844781d023ba27803af3f501296ff8243c2afdac76
● LMV	tokamak-network/ton-staking-v2	contracts/layer2/Layer2ManagerV1_1.sol	dc19b391b25154ec95ecd1722c381b53a23e164347234eccd3f2c93c7586c3ef
● OMV	tokamak-network/ton-staking-v2	contracts/layer2/OperatorManagerV1_1.sol	923efdbe24c7a0bfdd6b506bd9141ce80e85aa3c1ed17664b00b809f768839a5

ID	Repo	File	SHA256 Checksum
● DMS	tokamak-network/ton-staking-v2	contracts/stake/managers/DepositManagerV1_1Storage.sol	f443a0537c8d51762637aa059fd08f6bdc33b73d4ca08926c4fd19a45b72700
● SMS	tokamak-network/ton-staking-v2	contracts/stake/managers/SeigManagerV1_3Storage.sol	bbeabfb73ec2697ac2a9a333ba8f999f04dc016fbe0643f9ea19eac9147f0799
● NDA	tokamak-network/ton-staking-v2	2.deploy_NewDAOStructure.js	ebb6497bf82fdda577e37531bea9fe318b02a8fa70cbf27273d469e31ca7f84a

APPROACH & METHODS

TOKAMAK NETWORK - TON STAKING 2.5 AUDIT

This report has been prepared for Tokamak Network to discover issues and vulnerabilities in the source code of the Tokamak Network - TON Staking 2.5 Audit project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Formal Verification, Manual Review, and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

FINDINGS | TOKAMAK NETWORK - TON STAKING 2.5 AUDIT



15

Total Findings

0

Critical

2

Major

3

Medium

6

Minor

4

Informational

0

Discussion

This report has been prepared to discover issues and vulnerabilities for Tokamak Network - TON Staking 2.5 Audit. Through this audit, we have uncovered 15 issues ranging from different severity levels. Utilizing the techniques of Formal Verification, Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
CON-01	Centralization Risks	Centralization	Major	● Pending
CON-02	Potential Storage Collision	Design Issue	Major	● Pending
SMV-05	Unused Minted Tokens In <code>layer2Manager</code> Contract	Logical Issue	Medium	● Resolved
SMV-06	Incorrect Reward Allocation After Unpause	Logical Issue	Medium	● Resolved
SMV-07	Inconsistent Reward Calculation For Layer2	Logical Issue	Medium	● Resolved
CON-04	Third-Party Dependencies	Volatile Code	Minor	● Acknowledged
DAC-02	Unused Modifier	Coding Issue	Minor	● Resolved
DAC-03	Violation Of Check-Effect-Interaction Pattern	Concurrency	Minor	● Resolved
DAC-04	Ineffective <code>onlyMemberContract</code> Modifier	Volatile Code	Minor	● Resolved
DAO-01	Lack Of Storage Gap Or NameSpaced Storage Layout In Upgradeable Contract	Design Issue	Minor	● Acknowledged
DMV-01	<code>minDepositGasLimit</code> Is Not Used	Logical Issue	Minor	● Resolved

ID	Title	Category	Severity	Status
COT-01	Functions With <code>_</code> As Name Prefix Are Not <code>private</code> Or <code>internal</code>	Coding Style	Informational	Resolved
DAC-05	Redundant Code Components	Volatile Code	Informational	Resolved
DAC-06	Unused Internal Function	Coding Issue	Informational	Resolved
OMV-01	Typos	Coding Style	Informational	Resolved

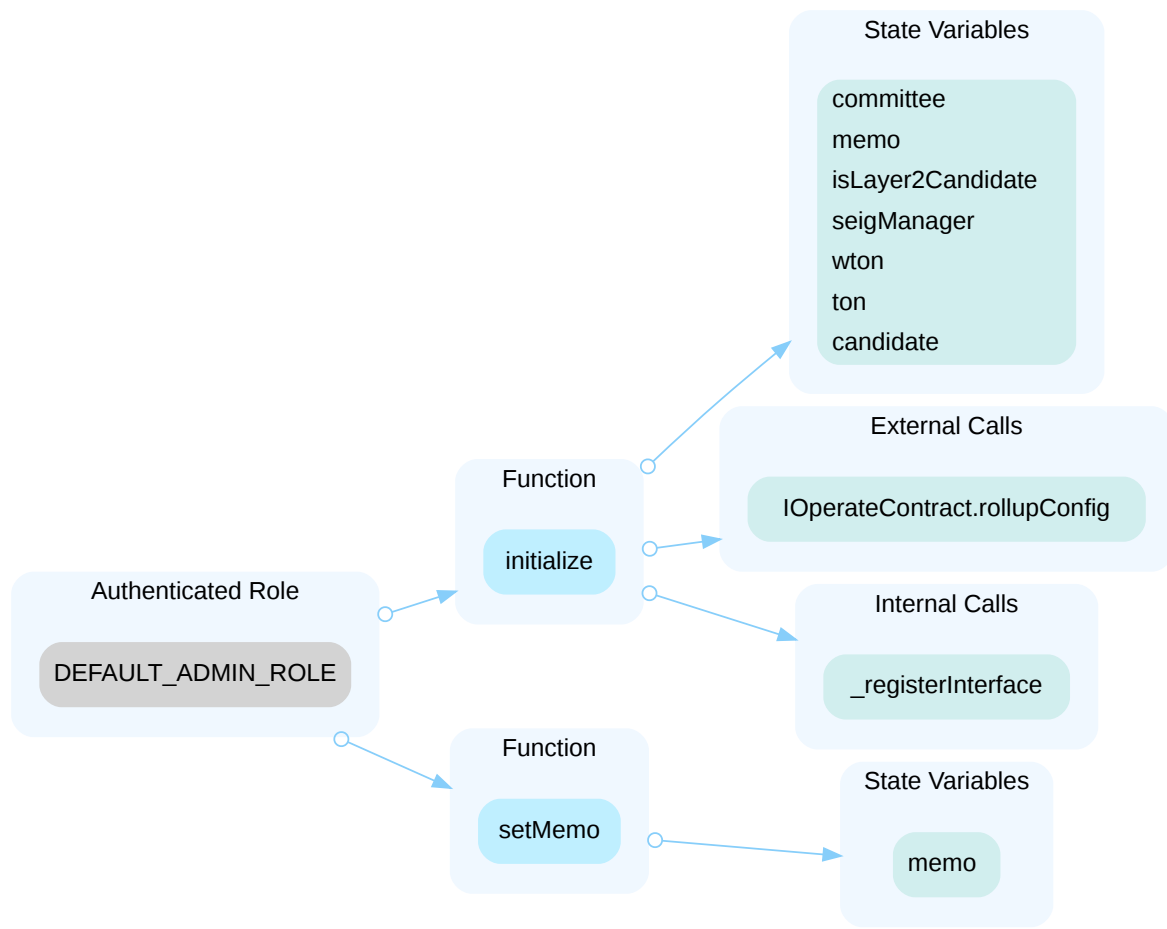
CON-01 | CENTRALIZATION RISKS

Category	Severity	Location	Status
Centralization	Major	contracts/dao/CandidateAddOnV1_1.sol (source); contracts/dao/DAOCommitteeOwner.sol (source); contracts/dao/DAOCommittee_V1.sol (source); contracts/layer2/L1BridgeRegistryV1_1.sol (source); contracts/layer2/Layer2ManagerV1_1.sol (source); contracts/layer2/OperatorManagerV1_1.sol (source); contracts/proxy/DAOCommitteeProxy2.sol (source); contracts/stake/managers/DepositManagerV1_1.sol (source); contracts/stake/managers/SeigManagerV1_3.sol (source)	Pending

Description

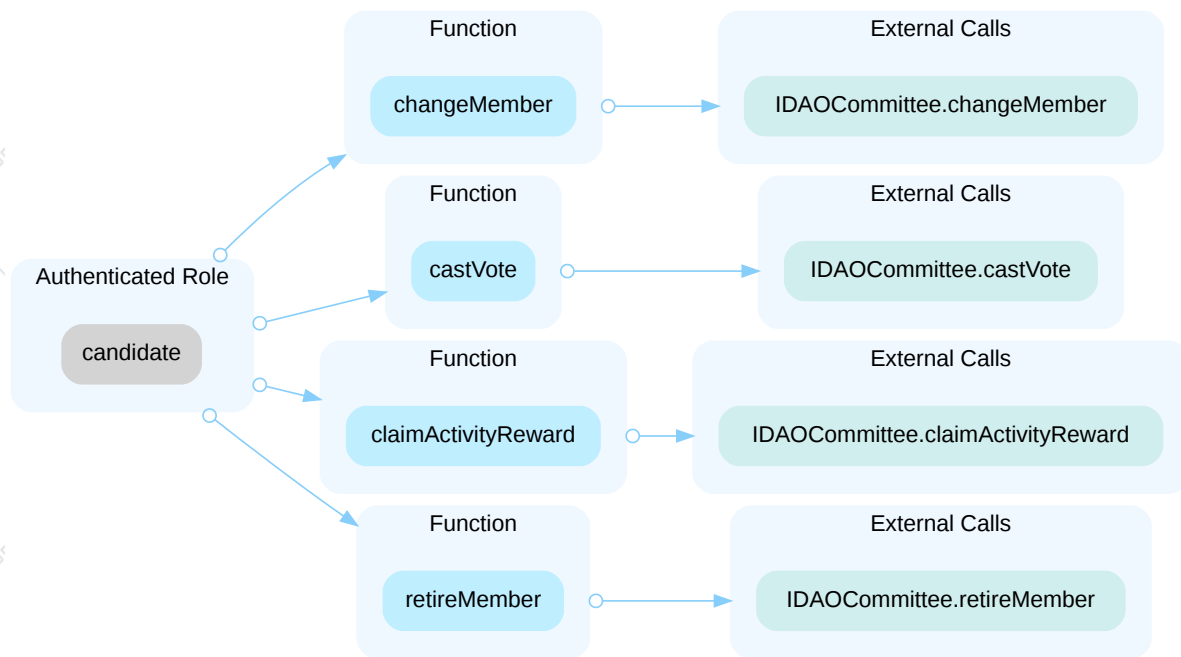
In the contract `CandidateAddOnV1_1` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

- function `initialize`, to set the state variables `candidate`, `isLayer2Candidate`, `committee`, `seigManager`, `memo`, `ton`, and `wton`.
- function `setMemo`, to set the state variable `memo`.



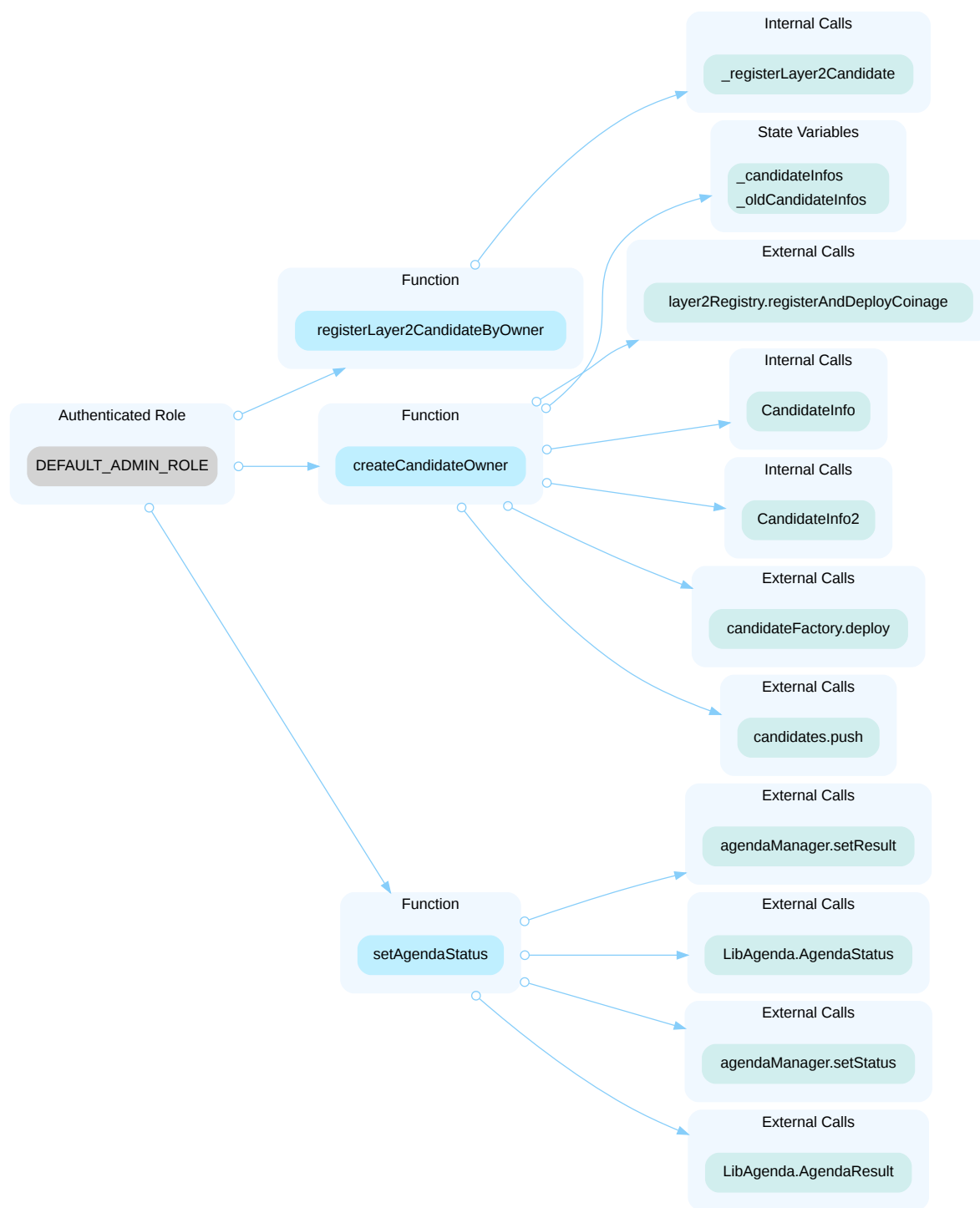
In the contract `CandidateAddOnV1_1` the role `candidate` has authority over the functions shown in the diagram below. Any compromise to the `candidate` account may allow the hacker to take advantage of this authority and:

- function `changeMember`, to become a member.
- function `retireMember`, to retire a member.
- function `castVote`, to vote on an agenda.
- function `claimActivityReward`, to claim an activity reward.



In the contract `DAOCCommittee_v1` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

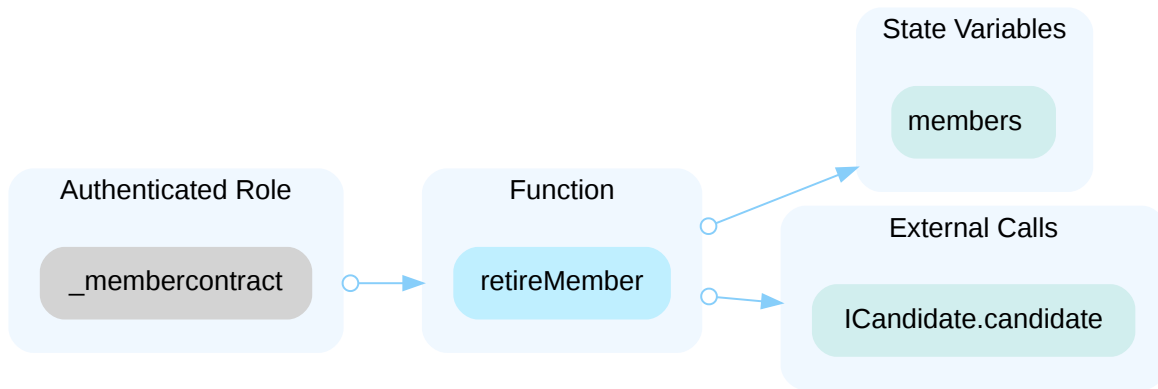
- function `createCandidateOwner` , to create a candidate.
- function `registerLayer2CandidateByOwner` , to registers the exist layer2 on DAO by owner.
- function `setAgendaStatus` , to set status and result of specific agenda.



In the contract `DAOCommittee_v1` the role `_membercontract` has authority over the functions shown in the diagram below.

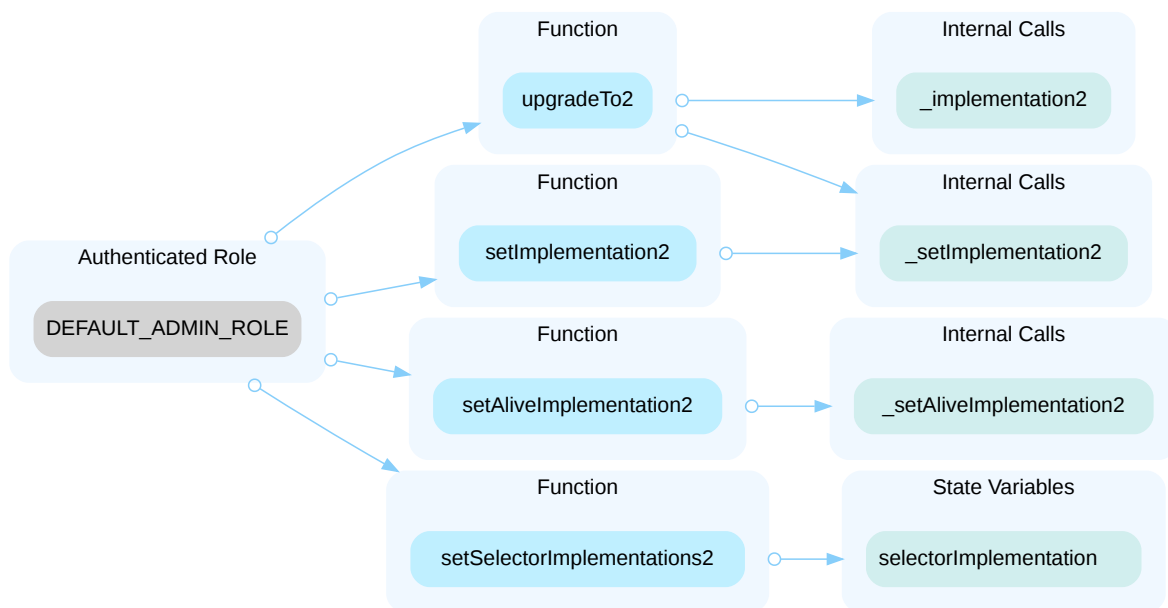
Any compromise to the `_membercontract` account may allow the hacker to take advantage of this authority and:

- function `retireMember`, to retire member.



In the contract `DAOCommitteeProxy2` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

- function `upgradeTo2`, to set implementation contract.
- function `setImplementation2`, to set implementation contract.
- function `setAliveImplementation2`, to implementation contract.
- function `setSelectorImplementations2`, to set `selectorImplementation`.



In the contract `DAOCommitteeOwner` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

- function `setCandidateAddOnFactory`, to set the `candidateAddOnFactory`.
- function `setLayer2Manager`, to set the `layer2Manager`.
- function `setTargetSetLayer2Manager`, to set the layer2 manager.
- function `setTargetSetL1BridgeRegistry`, to set the target L1 bridge registry.

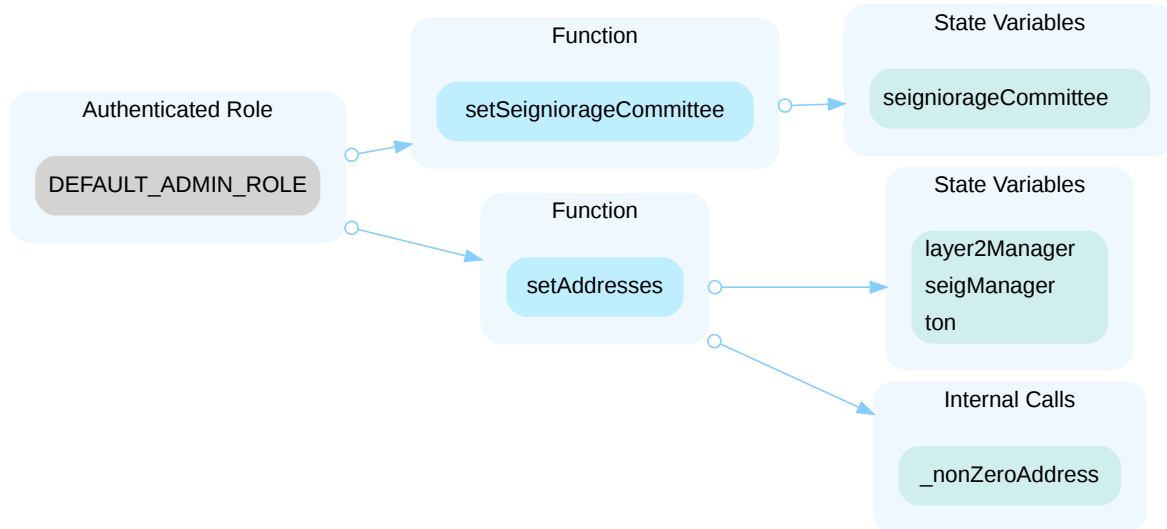
- function `setTargetLayer2StartBlock` , to set the layer2 start block.
- function `setTargetSetImplementation2` , to set the implementation contract.
- function `setTargetSetSelectorImplementations2` , to set the selector implementation.
- function `setSeigManager` , to set the seig manager.
- function `setTargetSeigManager` , to set the target seig manager.
- function `setSeigPause` , to set the seig manager paused.
- function `setSeigUnpause` , to set the seig manager unpause.
- function `setTargetGlobalWithdrawalDelay` , to set the global withdrawal delay.
- function `setTargetAddMinter` , to set the target add minter.
- function `setTargetUpgradeTo` , to upgrade logic.
- function `setTargetSetTON` , to set the TON addr.
- function `setTargetSetWTON` , to set the target WTON addr.
- function `setDaoVault` , to set the dao vault.
- function `setLayer2Registry` , to set the layer2 registry.
- function `setAgendaManager` , to set DAOAgendaManager contract address.
- function `setCandidateFactory` , to set CandidateFactory contract address.
- function `setTon` , to set the ton addr.
- function `setWton` , to set the wton addr.
- function `increaseMaxMember` , to increase the number of member slot.
- function `setQuorum` , to set new quorum.
- function `decreaseMaxMember` , to decreases the number of member slot.
- function `setActivityRewardPerSecond` , to set the `activityRewardPerSecond` .
- function `setCandidatesSeigManager` , to set SeigManager contract address on candidate contracts.
- function `setCandidatesCommittee` , to set DAOCommitteeProxy contract address on candidate contracts.
- function `setCreateAgendaFees` , to set fee amount of creating an agenda.

- function `setMinimumNoticePeriodSeconds` , to set the minimum notice period.
- function `setMinimumVotingPeriodSeconds` , to set the minimum voting period.
- function `setExecutingPeriodSeconds` , to set the executing period.
- function `setBurntAmountAtDAO` , to set the burnt amount at dao.



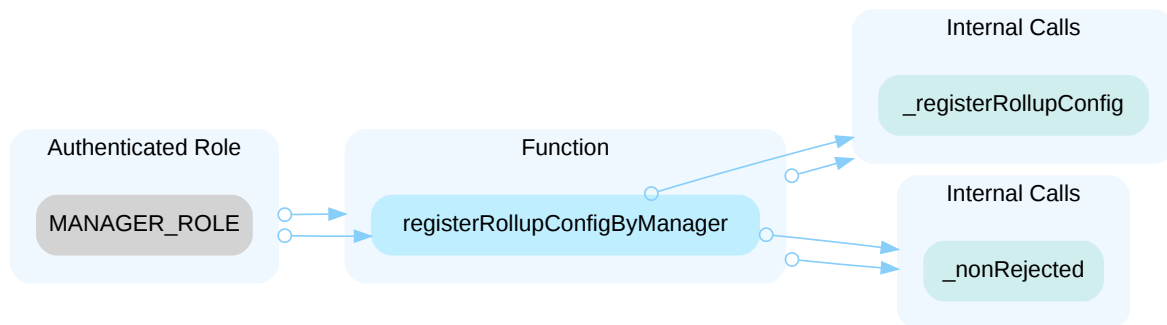
In the contract `L1BridgeRegistryV1_1` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

- function `setAddresses`, to set the state variables `layer2Manager`, `seigManager`, and `ton`.
- function `setSeigniorageCommittee`, to set the state variable `seigniorageCommittee`.



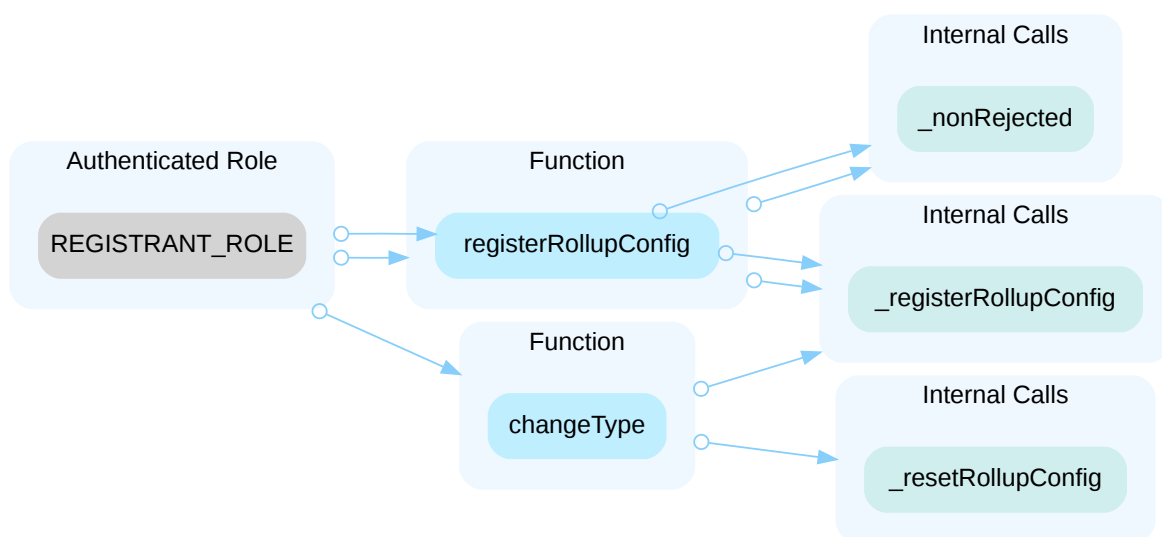
In the contract `L1BridgeRegistryV1_1` the role `MANAGER_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `MANAGER_ROLE` account may allow the hacker to take advantage of this authority and:

- function `registerRollupConfigByManager`, to registers Layer2 for a specific rollupConfig by the manager.



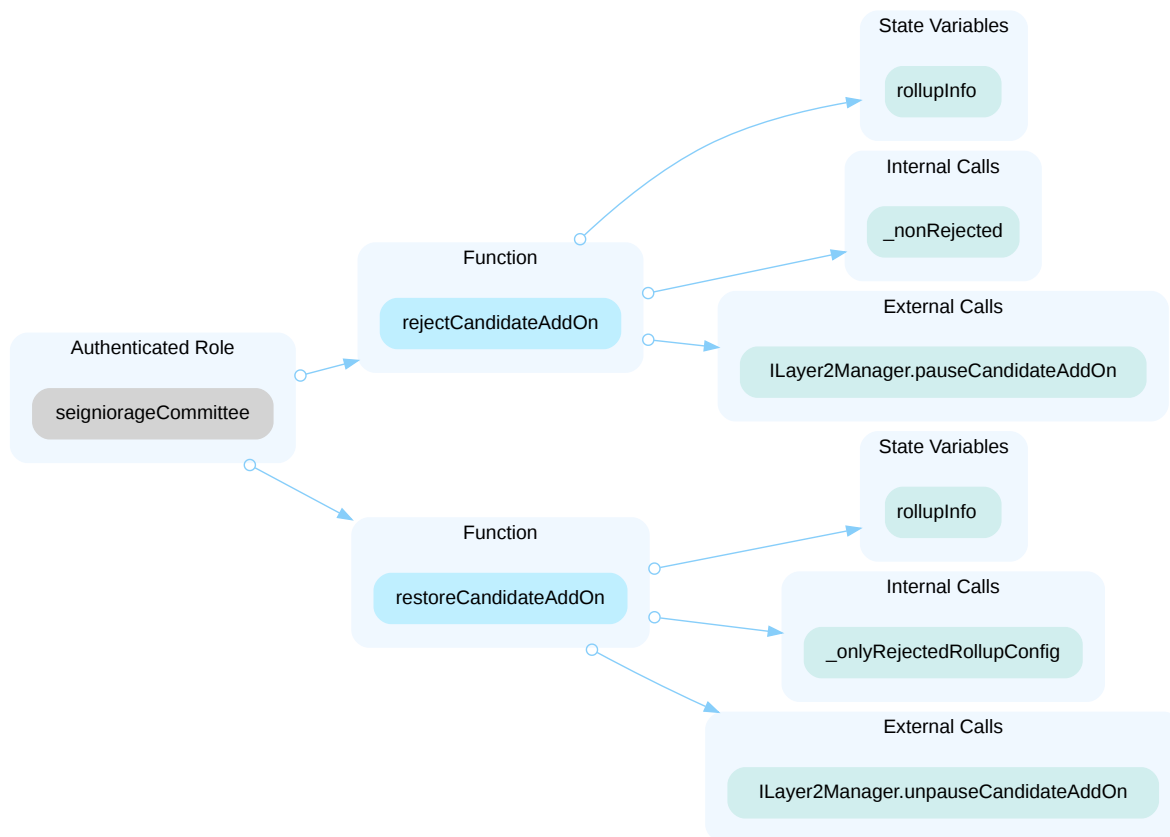
In the contract `L1BridgeRegistryV1_1` the role `REGISTRANT_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `REGISTRANT_ROLE` account may allow the hacker to take advantage of this authority and:

- function `registerRollupConfig`, to registers Layer2 for a specific rollupConfig by Registrant.
- function `changeType`, to change the Layer2 type for a specific rollupConfig by Registrant.



In the contract `L1BridgeRegistryV1_1` the role `seigniorageCommittee` has authority over the functions shown in the diagram below. Any compromise to the `seigniorageCommittee` account may allow the hacker to take advantage of this authority and:

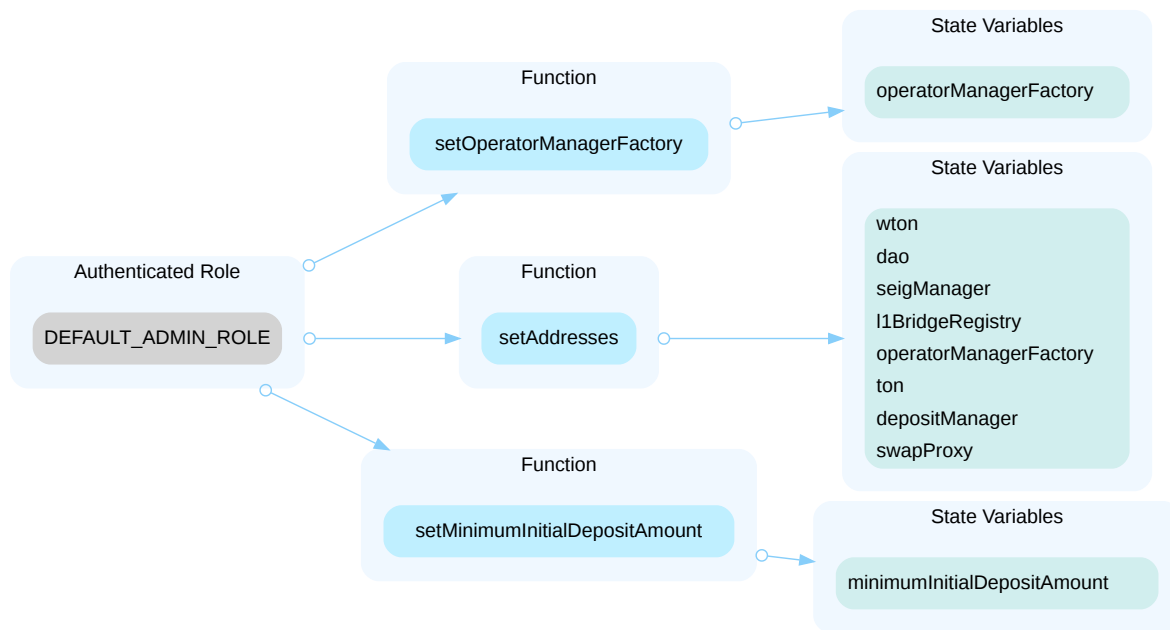
- function `rejectCandidateAddOn`, to stop issuing seigniorage to the layer 2 sequencer of a specific rollupConfig.
- function `restoreCandidateAddOn`, to restore cancel stopping seigniorage to the layer 2 sequencer of a specific rollupConfig.



In the contract `Layer2ManagerV1_1` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram

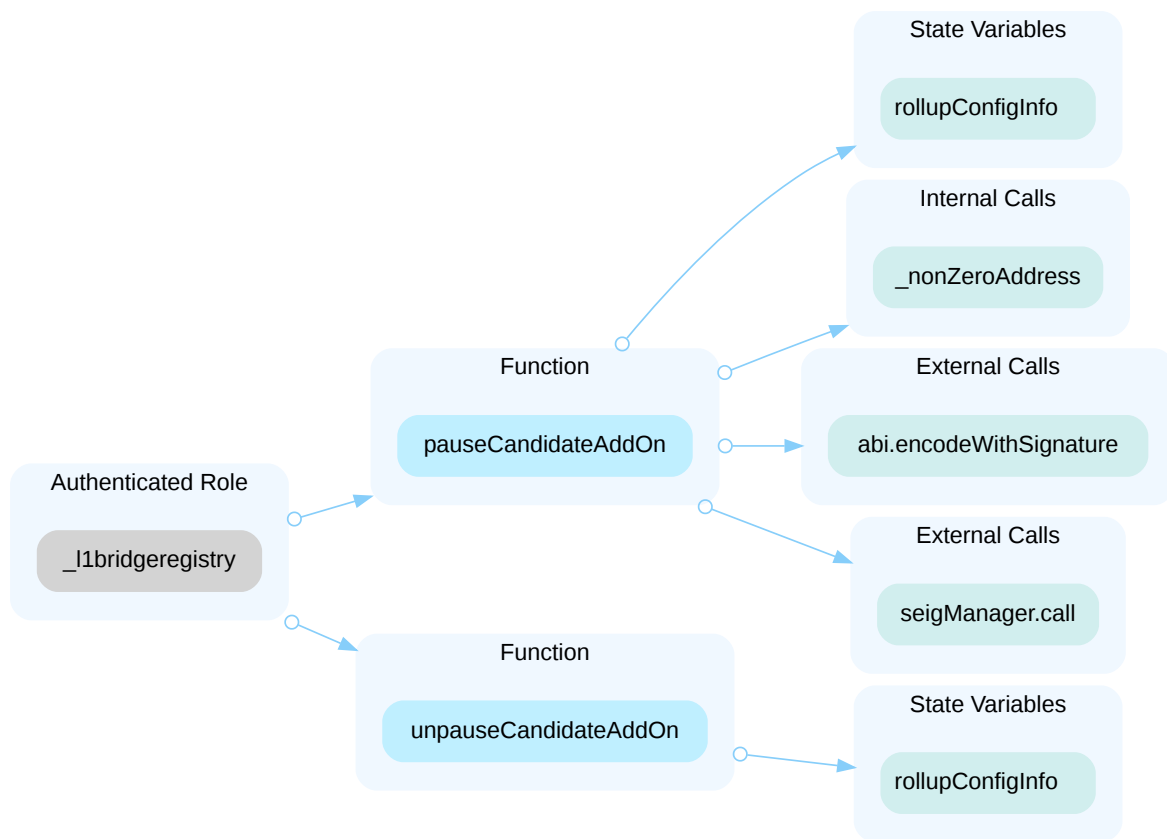
below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

- function `setAddresses` , to set the state variables `l1BridgeRegistry` , `operatorManagerFactory` , `ton` , `wton` , `dao` , `depositManager` , `seigManager` , and `swapProxy` .
- function `setOperatorManagerFactory` , to set the state variable `operatorManagerFactory` .
- function `setMinimumInitialDepositAmount` , to set the state variable `minimumInitialDepositAmount` .



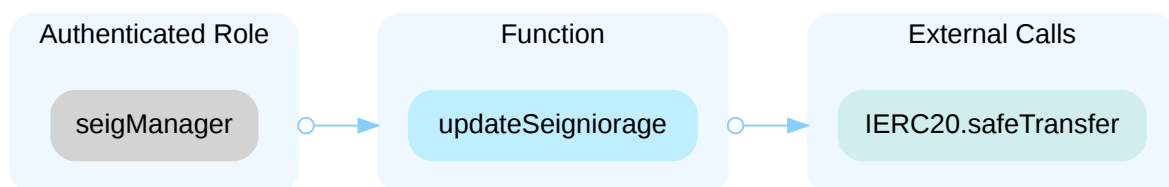
In the contract `Layer2ManagerV1_1` the role `l1bridgeregistry` has authority over the functions shown in the diagram below. Any compromise to the `l1bridgeregistry` account may allow the hacker to take advantage of this authority and:

- function `pauseCandidateAddOn` , to pause the `CandidateAddOn`.
- function `unpauseCandidateAddOn` , to unpause the `CandidateAddOn`.



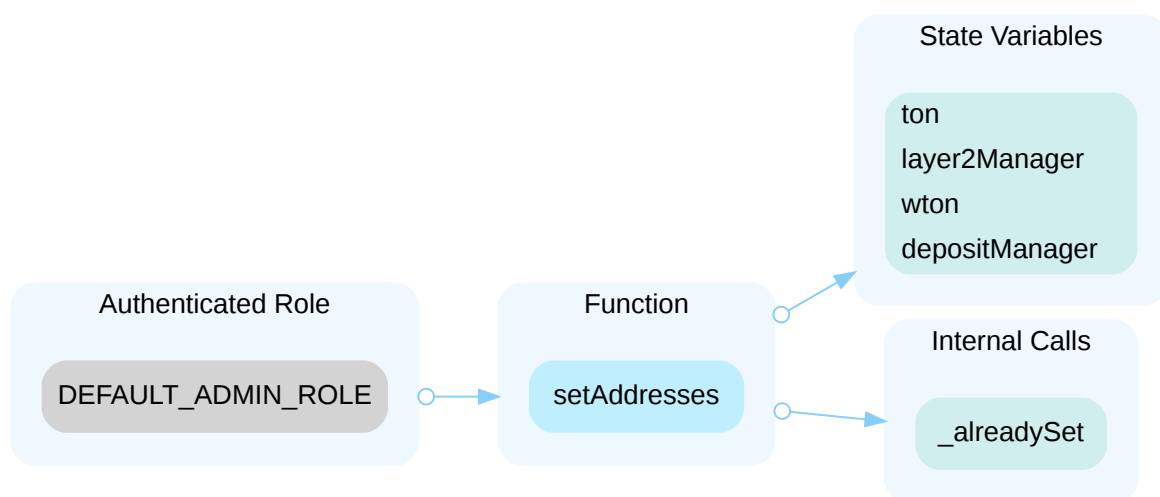
In the contract `Layer2ManagerV1_1` the role `seigManager` has authority over the functions shown in the diagram below. Any compromise to the `seigManager` account may allow the hacker to take advantage of this authority and:

- function `updateSeigniorage`, to settle the seigniorage to the Operator of Layer 2.



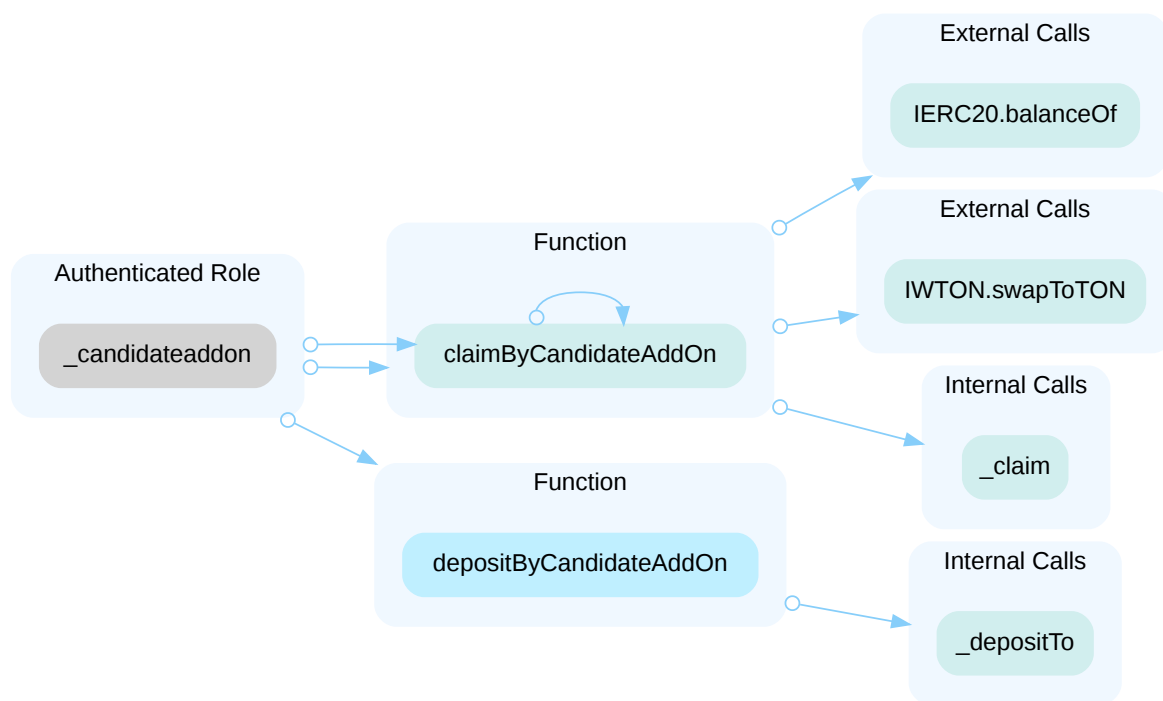
In the contract `OperatorManagerV1_1` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

- function `setAddresses`, to set the state variables `layer2Manager`, `depositManager`, `wton`, and `ton`.



In the contract `OperatorManagerV1_1` the role `_candidateaddon` has authority over the functions shown in the diagram below. Any compromise to the `_candidateaddon` account may allow the hacker to take advantage of this authority and:

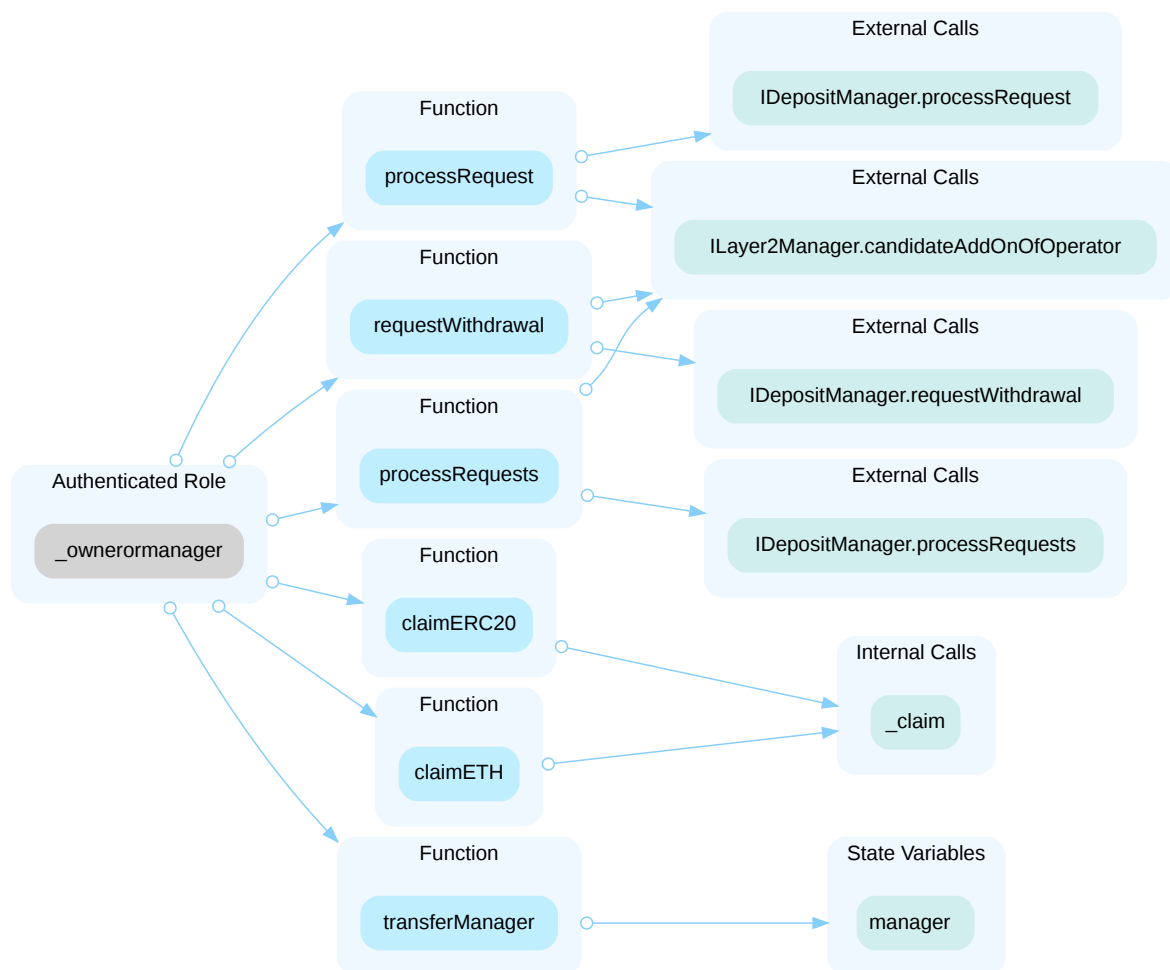
- function `depositByCandidateAddOn`, to deposit wton amount to DepositManager as named manager(EOA).
- function `claimByCandidateAddOn`, to claim WTON to a manager.



In the contract `OperatorManagerV1_1` the role `_ownerormanager` has authority over the functions shown in the diagram below. Any compromise to the `_ownerormanager` account may allow the hacker to take advantage of this authority and:

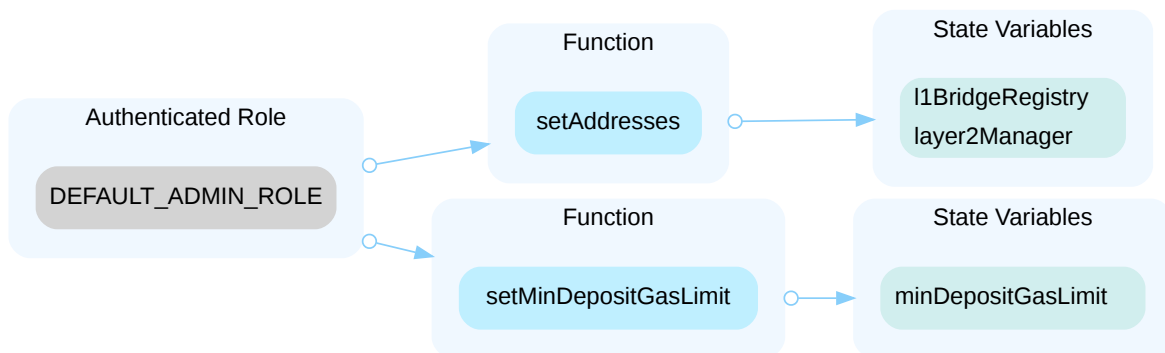
- function `transferManager`, to set the state variable `manager`.
- function `claimETH`, to give ETH to a manager.
- function `claimERC20`, to give ERC20 to a manager.
- function `requestWithdrawal`, to request withdrawal the staked ton.

- function `processRequest` , to process the withdrawal request.
- function `processRequests` , to process the withdrawal requests.



In the contract `DepositManagerV1_1` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

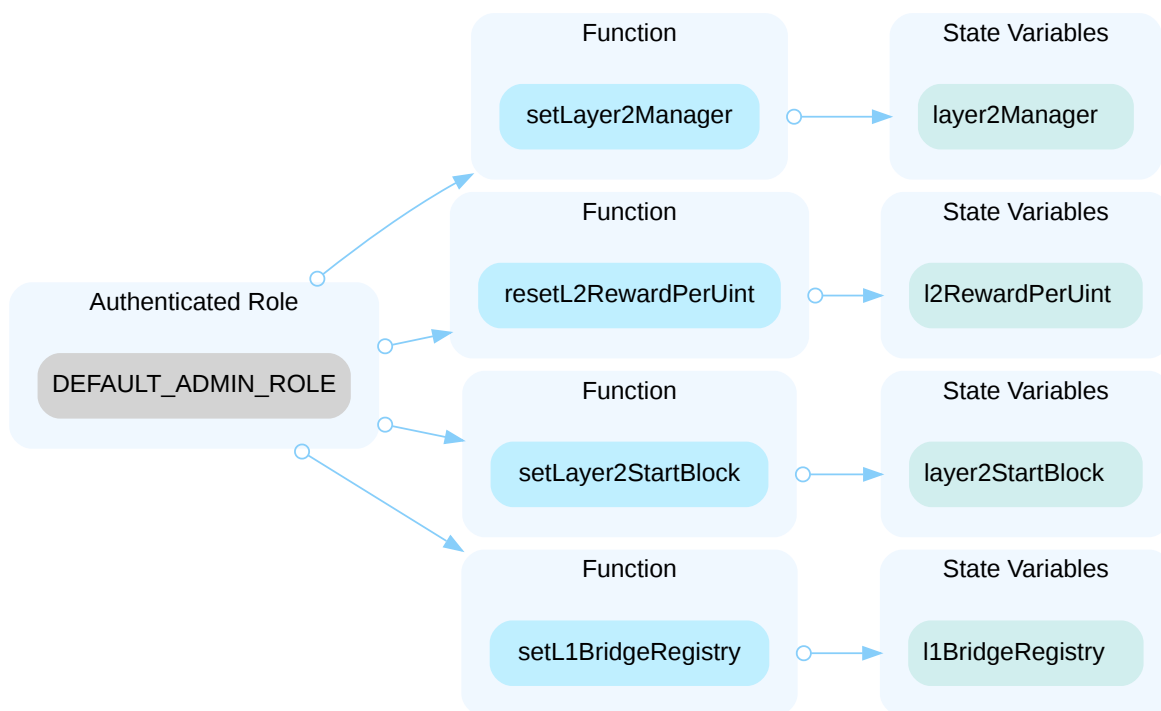
- function `setMinDepositGasLimit` , set the state variable `minDepositGasLimit` .
- function `setAddresses` , set the state variables `l1BridgeRegistry` and `layer2Manager` .



In the contract `SeigManagerV1_3` the role `DEFAULT_ADMIN_ROLE` has authority over the functions shown in the diagram

below. Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and:

- function `setLayer2Manager` , to set the state variable `layer2Manager` .
- function `setLayer2StartBlock` , to set the start block number of issuing a l2 seignorage.
- function `setL1BridgeRegistry` , to set the l1BridgeRegistry address.
- function `resetL2RewardPerUint` , to set the state variable `l2RewardPerUint` .



All the contracts inherit upgradeable contracts, indicating that it is part of an upgradeable system. Upgradeable contracts often pair with a proxy contract that is responsible for managing contract upgrades. The `privileged roles` of the proxy often have the authority to update the implementation contract. Any compromise of the privileged account could allow a hacker to exploit this authority, potentially altering the implementation contract pointed to by the proxy and thus executing malicious functionality within the implementation contract.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

CON-02 | POTENTIAL STORAGE COLLISION

Category	Severity	Location	Status
Design Issue	● Major	contracts/dao/DAOCommitteeOwner.sol (source): 56; contracts/dao/DAOCommittee_V1.sol (source): 56; contracts/dao/StorageStateCommitteeV2.sol (source): 56; contracts/proxy/ProxyStorage2.sol (source): 56	● Pending

Description

In solidity, the storage layout of a contract is determined by the inheritance order and the order in which variables are declared. Each variable has a fixed position (slot) in storage, starting from slot 0 and incrementing sequentially. When a contract inherits from multiple parent contracts, the storage variables of the parent contracts are arranged in the order of inheritance.

The `DAOCommittee_V1` contract originally only inherits from `StorageStateCommitteeV2` contract, refer to [this address](#). In the upgraded version, it inherits from a new contract, `ProxyStorage2`, and place `ProxyStorage2` before `StorageStateCommitteeV2`. This change in inheritance order will alter the storage layout, potentially causing storage collisions.

Analysis of Storage Collisions

In the original layout, the `_implementation` variable is at slot 0, but in the new layout, it is at slot 3. This means that the `_implementation` variable, which was originally stored at slot 0, will now be overwritten by `proxyImplementation` (mapping) in the new layout. This change in storage layout causes misalignment of data. Data that was originally stored in `_implementation` will now be misinterpreted as data for `proxyImplementation`, which leads to contract logic errors or data corruption.

The same issue exists in the `DAOCommitteeOwner` contract.

Recommendation

Keep inheritance order unchanged: place `ProxyStorage2` after `StorageStateCommitteeV2` to preserve the original storage layout.

SMV-05 | UNUSED MINTED TOKENS IN layer2Manager CONTRACT

Category	Severity	Location	Status
Logical Issue	Medium	contracts/stake/managers/SeigManagerV1_3.sol (source): 636	Resolved

Description

The `_increaseTot()` function is designed to distribute rewards to multiple entities, including the user, powerton, and the dao. However, a critical issue arises at line 636, where the function mints wton tokens to the layer2Manager contract. When the layer2Allowed variable is set to false, the minted tokens remain trapped in the layer2Manager contract, as it lacks the functionality to withdraw them. This results in an unallocated mint of tokens, creating a potential loss of funds or unintended token distribution.

```
614 (rollupConfig, layer2Allowed) = allowIssuanceLayer2Seigs(msg.sender);
```

Additionally, if the layer2 address associated with each call to `_increaseTot()` is not permitted to receive rewards (as determined by the `allowIssuanceLayer2Seigs` function at line 614), the layer2Allowed variable will be set to false. In this case, the `l2RewardPerUnit` value will not accrue, and other allowable rewards will also fail to be distributed. This behavior is problematic as it disrupts the intended reward distribution mechanism.

SeigManagerV1_3.sol

```
636 if (l2TotalSeigs != 0) IWTON(wton_).mint(_layer2Manager, l2TotalSeigs);
```

```
654     if (layer2Allowed) {
655         if (l2TotalSeigs != 0) l2RewardPerUint += ((l2TotalSeigs * 1e18) /
totalLayer2TVL);
656
657         Layer2Reward storage newLayer2Info = layer2RewardInfo[msg.sender];
658
659         if (l2RewardPerUint != 0) {
660             if (_isSenderOperator || oldLayer2Info.layer2Tv1 > curLayer2Tv1
) {
661                 layer2Seigs = unsettledReward(msg.sender);
662
663                 if (layer2Seigs != 0) {
664                     ILayer2Manager(_layer2Manager).updateSeigniorage(
rollupConfig, layer2Seigs);
665                     newLayer2Info.initialDebt += layer2Seigs;
666                 }
667                 } else if (_lastCommitBlock[msg.sender] == 0) {
668                     newLayer2Info.initialDebt = (l2RewardPerUint *
oldLayer2Info.layer2Tv1) / 1e18;
669                 }
670             }
671
672             newLayer2Info.layer2Tv1 = curLayer2Tv1;
673             totalLayer2TVL = totalLayer2TVL + curLayer2Tv1 - oldLayer2Info.
layer2Tv1;
674         }
```

Recommendation

Add proper checks and conditions to prevent tokens from being minted unnecessarily or being locked in the contract.

Alleviation

[Tokamak Network Team, 02/27/2025]: The team heeded the advice and resolved the issue in commit:

8bebb4fe6c6238156bf42b2bb087e26a1f6b7384.

SMV-06 | INCORRECT REWARD ALLOCATION AFTER UNPAUSE

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/stake/managers/SeigManagerV1_3.sol (source): 659	● Resolved

Description

The `_increaseTot()` function is responsible for distributing rewards to various roles. At line 661, the function calculates the reward tokens for layer2. When rewards for layer2 are paused and later resumed, the variable `initialDebt` is set to 0. As a result, layer2 is incorrectly allowed to claim rewards for the paused period, leading to undesired reward distribution.

```
192     function excludeFromSeigniorage(address _layer2) external returns (bool) {
193         _onlyLayer2Manager();
194         Layer2Reward storage reward = layer2RewardInfo[_layer2];
195         // require (totalLayer2TVL >= reward.layer2Tvl, "check layer2Tvl");
196         if (totalLayer2TVL < reward.layer2Tvl) revert Layer2TvlError();
197
198         emit ExcludedFromSeigniorage(_layer2, reward.layer2Tvl, reward.
initialDebt);
199
200         if (reward.layer2Tvl != 0) {
201             totalLayer2TVL -= reward.layer2Tvl;
202             reward.layer2Tvl = 0;
203             reward.initialDebt = 0;
204         }
205
206         return true;
207     }
```

```
374     function unSettledReward(address layer2) public view returns (uint256
amount) {
375         Layer2Reward memory layer2Info = layer2RewardInfo[layer2];
376         if (layer2Info.layer2Tvl != 0)
377             amount = l2RewardPerUint * (layer2Info.layer2Tvl / 1e18) -
layer2Info.initialDebt;
378     }
```

```
654     if (layer2Allowed) {
655         if (l2TotalSeigs != 0) l2RewardPerUint += ((l2TotalSeigs * 1e18) /
totalLayer2TVL);
656
657         Layer2Reward storage newLayer2Info = layer2RewardInfo[msg.sender];
658
659         if (l2RewardPerUint != 0) {
660             if (_isSenderOperator || oldLayer2Info.layer2Tvl > curLayer2Tvl
) {
661                 layer2Seigs = unSettledReward(msg.sender);
662
663                 if (layer2Seigs != 0) {
664                     ILayer2Manager(_layer2Manager).updateSeigniorage(
rollupConfig, layer2Seigs);
665                     newLayer2Info.initialDebt += layer2Seigs;
666                 }
667                 } else if (_lastCommitBlock[msg.sender] == 0) {
668                     newLayer2Info.initialDebt = (l2RewardPerUint *
oldLayer2Info.layer2Tvl) / 1e18;
669                 }
670             }
671
672             newLayer2Info.layer2Tvl = curLayer2Tvl;
673             totalLayer2TVL = totalLayer2TVL + curLayer2Tvl - oldLayer2Info.
layer2Tvl;
674         }
```

Recommendation

Ensure that initialDebt is correctly set when resuming rewards after a pause. The function should include checks to prevent layer2 from claiming rewards for the paused period unless explicitly intended.

Alleviation

[Tokamak Network Team, 02/27/2025]: The team heeded the advice and resolved the issue in commits:

`27e4f2642c9f39c14eb8e4ac56e14d8bbdc3981d` and `27e4f2642c9f39c14eb8e4ac56e14d8bbdc3981d` .

SMV-07 | INCONSISTENT REWARD CALCULATION FOR LAYER2

Category	Severity	Location	Status
Logical Issue	Medium	contracts/stake/managers/SeigManagerV1_3.sol (source): 659	Resolved

Description

The `_increaseTot()` function is used to distribute rewards to various roles. At line 659, the reward calculation for layer2 is inconsistent. If it's the first time layer2 is calling `_increaseTot`, and the check at line 660 returns true, the reward calculation starts with `initialDebt` equal to 0. If the check returns false, the calculation starts with `initialDebt` equal to $(l2RewardPerUnit * oldLayer2Info.layer2Tv1) / 1e18$. This inconsistency may lead to unexpected reward distribution.

```
659         if (l2RewardPerUnit != 0) {
660             if (_isSenderOperator || oldLayer2Info.layer2Tv1 > curLayer2Tv1) {
661                 layer2Seigs = unsettledReward(msg.sender);
662             }
663             if (layer2Seigs != 0) {
664                 ILayer2Manager(_layer2Manager).updateSeigniorage(rollupConfig, layer2Seigs);
665                 newLayer2Info.initialDebt += layer2Seigs;
666             }
667             } else if (_lastCommitBlock[msg.sender] == 0) {
668                 newLayer2Info.initialDebt = (l2RewardPerUnit *
oldLayer2Info.layer2Tv1) / 1e18;
669             }
670         }
```

Recommendation

Ensure that the logic for `initialDebt` is consistently handled for all layer2 calculations.

Alleviation

[Tokamak Network Team, 02/27/2025]: The team heeded the advice and resolved the issue in commit: 27e4f2642c9f39c14eb8e4ac56e14d8bbdc3981d.

CON-04 | THIRD-PARTY DEPENDENCIES

Category	Severity	Location	Status
Volatile Code	Minor	contracts/layer2/OperatorManagerV1_1.sol (source): 23; contracts/stake/managers/DepositManagerV1_1.sol (source): 174~200	Acknowledged

Description

The contract is serving as the underlying entity to interact with third-party or out of scope contracts, such as `IL1Bridge`, `IRollupConfig`, etc.. The scope of the audit treats third-party and out of scope entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets.

Recommendation

We recommend that the project team constantly monitor the functionality of third-party and out of scope dependencies to mitigate any side effects that may occur when unexpected changes are introduced.

Alleviation

[Tokamak Network Team, 02/26/2025]: The team acknowledged the finding and decided not to change the current codebase.

Our efforts to avoid dependencies due to registering RollupConfig (Third-Party)

When registering RollupConfig, code verification of the Third-Party L1 contract is performed.

When registering L2 RollupConfig information (`L1BridgeRegistryV1_1.registerRollupConfig`), Third-Party contract information is stored in the L2 RollupConfig contract.

This registration function can only be executed by the registrant.

The registrant will go through the verification process below to register the L2 RollupConfig contract.

- Code hash of the proxy of the Third-Party L1 contracts,
- Code hash of the logic of the Third-Party L1 contracts,
- Code hash of the proxy and logic of ProxyAdmin (of Third-Party L1 contracts),
- Code hash of the proxy and logic of the safe wallet contract which is ProxyAdmin's owner.
- And, verification of the accounts with the authority of the safe wallet contract . (These accounts will consist of DAO, Foundation, and L2 operator.)

This registration will be done according to the user's choice when deploying L2 (e.g. Thanos stack) using Tokamak Rollup Hub, and then distribution will be done after L2 verification of TRH (Tokamak Rollup Hub).

Therefore We will target the RollupConfig of L2 published by TRH and ensure verification of this code by TRH(Tokamak Rollup Hub).

■ The user's effort to verify L2 is required before using the 'Withdrawal and Deposit (L2)' function

When withdrawing staking and trying to deposit to a third-party service (DepositManager.withdrawAndDepositL2 function), the user is required to make efforts to review the safety of the third-party service. The user must move to the third-party service and take responsibility for any damages incurred from the third-party service. Therefore, the user must use the function after verifying and confirming the third-party service.

The reason why this function is provided despite such efforts is because the 'withdrawal and deposit (to L2)' function is a very attractive function for L2 users.

The withdrawal service requires a withdrawal waiting time of about 2 weeks after the withdrawal request. However, the function of depositing to the corresponding L2 at the same time as the withdrawal through the DepositManager.withdrawAndDepositL2 function is a very beneficial function for L2 users because it deposits to L2 at the same time as the withdrawal without a withdrawal waiting time. Therefore, if the effort to confirm the user's L2 is involved, this function will be very convenient and beneficial to the user.

DAC-02 | UNUSED MODIFIER

Category	Severity	Location	Status
Coding Issue	Minor	contracts/dao/DAOCommittee_V1.sol (source): 148~151	Resolved

Description

The `DAOCommittee_V1` contract contains a `onlyLayer2Manager` modifier, but the modifier is never used in the contract.

Recommendation

Consider utilizing the modifier where needed, or removing unused modifiers.

Alleviation

[Tokamak Network Team, 03/03/2025]: The team heeded the advice and resolved the issue in commit: `60fcdddde9131e487953c40fc1496f8bd2f9568f`.

DAC-03 | VIOLATION OF CHECK-EFFECT-INTERACTION PATTERN

Category	Severity	Location	Status
Concurrency	Minor	contracts/dao/DAOCommittee_V1.sol (source): 550-566	Resolved

Description

Check-effect-interaction pattern violation occurs when a function makes an external call to another contract before resolving any effects. If an attacker can control the contract, they can make a recursive call back to the original function, repeating interactions that would have otherwise not run after the external call resolved the effects.

In the `claimActivityReward()` function, the state variables update takes place after the external call.

Recommendation

We recommend using the [Checks-Effects-Interactions Pattern](#) to avoid the risk of calling external contracts.

Alleviation

[Tokamak Network Team, 02/27/2025]: The team heeded the advice and resolved the issue in commit: 275be9b7d3ac9feaf69a3a0d082ab9a78ddc9997.

DAC-04 | INEFFECTIVE `onlyMemberContract` MODIFIER

Category	Severity	Location	Status
Volatile Code	Minor	contracts/dao/DAOCommittee_V1.sol (source): 360	Resolved

Description

The `onlyMemberContract` modifier has the following implementation. A malicious `msg.sender` contract can implement arbitrary `candidate()` function to return a valid member address and pass the check, despite not being a member contract. In the `retireMember()` function, the impact is limited due to the additional `require` statement in line 363-366, but it should be noted that the modifier itself is insufficient, and can have potentially dangerous implications if a future upgrade include the usage of this modifier without additional input validation.

```
modifier onlyMemberContract() {  
    address candidate = ICandidate(msg.sender).candidate();  
    require(isMember(candidate), "StorageStateCommittee: not a member");  
    _;  
}
```

Recommendation

Consider including additional input validation in the modifier to ensure that the `msg.sender` is indeed a valid member contract.

Alleviation

[Tokamak Network Team, 02/28/2025]: The team heeded the advice and resolved the issue in commit: 13e3c030e22b09eb9df6cc230dd0b3130fa2c533.

DAO-01 | LACK OF STORAGE GAP OR NAMESPACE STORAGE LAYOUT IN UPGRADEABLE CONTRACT

Category	Severity	Location	Status
Design Issue	Minor	contracts/dao/CandidateAddOnV1_1.sol (source); contracts/dao/DAOCommitteeOwner.sol (source); contracts/dao/DAOCommittee_V1.sol (source); contracts/layer2/L1BridgeRegistryV1_1.sol (source); contracts/layer2/Layer2ManagerV1_1.sol (source); contracts/layer2/OperatorManagerV1_1.sol (source); contracts/stake/managers/DepositManagerV1_1.sol (source); contracts/stake/managers/SeigManagerV1_3.sol (source)	Acknowledged

Description

When updating upgradeable smart contracts for new features or bug fixes, keeping the state variables' declaration order unchanged is essential to avoid storage layout issues.

A practical solution is to include unused state variables or explicitly named storage gaps (like `__gap`) in the base contracts. This foresight allows reserved slots for future use, ensuring that any additions to the contract's state won't disrupt the storage pattern of derived contracts or the compatibility with previously deployed versions. After [ERC-7201](#), it is also possible to place all storage variables of a contract into one or more structs like [Namespaced Storage Layout](#).

The problem of "Lack of Storage Gap Or NameSpaced Storage Layout in Upgradeable Contract" occurs when **these storage gaps are not incorporated into the base contract's logic nor the base contract defines the namespace storage layout**. As a result, if new state variables are added to the base contract, they might overwrite existing

Recommendation

To mitigate this issue:

1. To ensure flexibility for future upgrades to the logic contract, it is recommended to reserve an appropriate storage gap in the base contract. This can be achieved by declaring a fixed-size array, typically of `uint256` elements (each occupying a 32-byte slot), within the base contract. Use a clear identifier, such as `__gap` or a name prefixed with `__gap_`, to explicitly indicate its purpose as reserved storage space.
2. Alternatively, it is recommended to organize all storage variables in the contract into one or more structs, following a pattern such as [Namespaced Storage Layout](#), to streamline management and maintain compatibility across upgrades.

More detailed info :

- https://docs.openzeppelin.com/contracts/4.x/upgradeable#storage_gaps

I Alleviation

[Tokamak Network Team, 02/25/2025]:

The team acknowledged the issue and decided to remain unchanged in the scope of the audit. Moreover, the explanation of the proxy used by this service is here: [link](#)

These contracts (SeigManagerProxy, DepositManagerProxy, DAOCommitteeProxy) are already used in the service. the deployed addresses on mainnet. Therefore, reconfiguring these proxies(SeigManagerProxy, DepositManagerProxy, DAOCommitteeProxy) is out of the scope of this audit.

All logic shares the same memory. When we configure the upgrade logic, we configure the storage to exactly match the storage layout of the previous version, and create additional storage. We will not tolerate any storage mismatch because we must maintain the existing service. Therefore, we will not introduce the concept of a storage gap.

DMV-01 | minDepositGasLimit IS NOT USED

Category	Severity	Location	Status
Logical Issue	Minor	contracts/stake/managers/DepositManagerV1_1.sol (source): 103~106, 157~158, 180, 195, 204	Resolved

Description

The `DepositManagerV1_1` contract contains a `minDepositGasLimit` setter function, but the value of `minDepositGasLimit` is never utilized outside of the setter function. In the `withdrawAndDepositL2()` function, a different local variable `_minDepositGasLimit` is declared and utilized instead.

Recommendation

Consider removing unused setter function and / or update the logic in the `withdrawAndDepositL2()` function to use the `minDepositGasLimit` variable instead.

Alleviation

[Tokamak Network Team, 02/25/2025]: The team heeded the advice and resolved the issue in commit: `a18a68f29fdde3bb29aeebba4df2e2e28ee0ee4a`.

COT-01 | FUNCTIONS WITH `_` AS NAME PREFIX ARE NOT `private` OR `internal`

Category	Severity	Location	Status
Coding Style	● Informational	DAOCommittee_V1.sol (69556b8): 568-570; Layer2ManagerV1_1.sol (69556b8): 405-435	● Resolved

Description

Functions with names starting with `_` should be declared as `private` / `internal`.

Recommendation

Consider changing function visibility to private or removing `_` from the start of the function name.

Alleviation

[Tokamak Network Team, 02/27/2025]: The team heeded the advice and resolved the issue in commit:

05c0080d0fe4177de7d3304173ea0f37222bb5d8.

DAC-05 | REDUNDANT CODE COMPONENTS

Category	Severity	Location	Status
Volatile Code	● Informational	contracts/dao/DAOCommittee_V1.sol (source): 621-624	● Resolved

Description

The below two `require` statements check the same logic, hence one of them is redundant and does not affect the functionality of the codebase.

```
function _registerLayer2Candidate(address _operator, address _layer2, string memory _memo){
    require(!isExistCandidate(_layer2), "DAOCommittee: candidate already registered");
    ...
    require(
        _candidateInfos[_layer2].candidateContract == address(0),
        "DAOCommittee: The candidate already has contract"
    );
    ...
}

function isExistCandidate(address _candidate) public view returns (bool isExist) {
    return _candidateInfos[_candidate].candidateContract != address(0);
}
```

Recommendation

We advise to remove the redundant statements for production environments.

Alleviation

[Tokamak Network Team, 02/28/2025]: The team heeded the advice and resolved the issue in commit: [c148a05dfe1a0d0e1506d6857213cfc9bebed850](#).

DAC-06 | UNUSED INTERNAL FUNCTION

Category	Severity	Location	Status
Coding Issue	● Informational	contracts/dao/DAOCommittee_V1.sol (source): 696~704	● Resolved

Description

The `_call()` function is `internal` and is never used in the contract.

Recommendation

We recommend either removing or using the internal function.

Alleviation

[Tokamak Network Team, 02/27/2025]: The team heeded the advice and resolved the issue in commit: [2e96e1b6e6deae29c60d3bb10944c86eb0ae296b](#).

OMV-01 | TYPOS

Category	Severity	Location	Status
Coding Style	● Informational	contracts/layer2/OperatorManagerV1_1.sol (source): 221~223	● Resolved

Description

The variable `fa1gTon` should be spelled `f1agTon` as it is a flag that indicates if Ton or wTon should be used.

Recommendation

We recommend fixing all the typos in the contract for readability.

Alleviation

[Tokamak Network Team, 02/27/2025]: The team heeded the advice and resolved the issue in commit:

f315b0996b4a7a408cdf296ce613d071749c3d8b.

APPENDIX | TOKAMAK NETWORK - TON STAKING 2.5 AUDIT

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Concurrency	Concurrency findings are about issues that cause unexpected or unsafe interleaving of code executions.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Entire **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.



Tokamak Network - TON Staking 2.5 Audit Preliminary
Comments

CertiK Assessed on Mar 4th,
2025

Copyright ©
CertiK